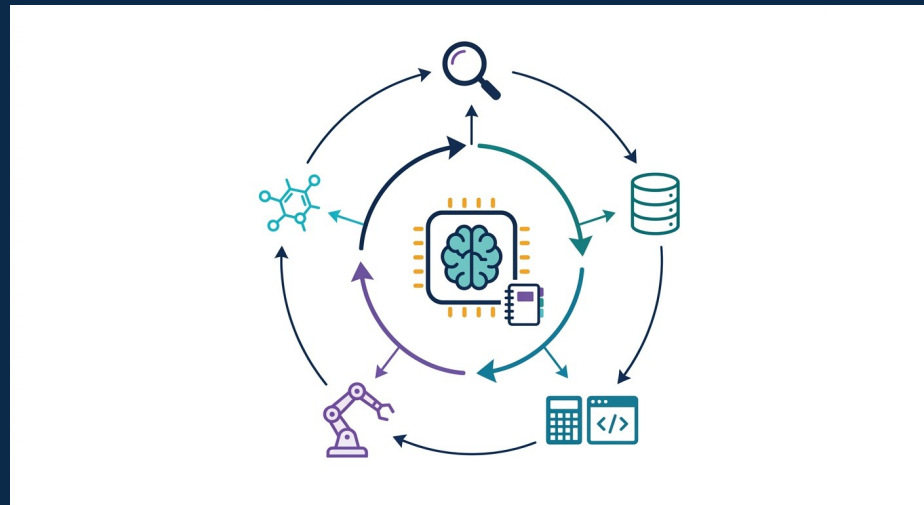




6일차 · 3교시 — 스스로 도구를 쓰게: LLM 에이전트

원리(ReAct·tool·계획·메모리·multi-agent) · 사례(ChemCrow·Coscientist·AI co-scientist·Robin·Claude for LS)

활용을 넘어, 스스로 일하게

2교시까지 (활용·신뢰)

- LLM에 '프롬프트'로 시켜 답을 받음
- RAG로 근거를 붙이고 검증
- 하지만 사람이 매번 지시·도구 실행

3교시 (에이전트)

- LLM이 '스스로' 도구를 골라 실행
- 생각→행동→관찰을 반복(ReAct)
- 문헌·검색·계산·실험을 자율 오케스트레이션
DMTA 자동화

+심화

핵심 전환: '내가 시키는 LLM'(2교시) → 'LLM이 도구를 스스로 쓰는 에이전트'(3교시). 단일 LLM의 한계(계산·최신성·외부세계)를 '도구+계획+검증'으로 넘어선다.

3교시에서 볼 것

A. 에이전트 원리 (심화)

단일 LLM 한계 · think-act-observe · RAG · ReAct · Toolformer · 계획 · 메모리
· reflection · multi-agent

B. 신약개발 에이전트 사례

초기(ChemCrow · Coscientist) → 2025-26 다중에이전트(AI co-scientist · TxAgent · PharmAgents · DiscoVerse) · Claude for LS

C. 자율 워크플로우 · 평가 · 한계

DMTA 페루프 · 자율실험실 · 벤치마크 · 오류전파 · 검증 · 보안 예고

쉽게

오늘은 '에이전트가 무엇이며 어떻게 작동하는가'(A)를 단단히 잡고, 신약개발에서 '실제로 무엇을 해냈는가'(B)를 사례로 확인한 뒤, '자율 워크플로우와 그 한계'(C)로 마무리합니다.

PART A

에이전트 원리 (심화)

LLM(추론) + 도구 + 계획 + 메모리 + 검증

왜 단일 LLM이 아니라 에이전트인가

단일 LLM 한계

- 최신성 결여 — 학습 시점에 지식이 고정
- 정밀 계산·구조식·수식 처리 취약
- 환각(그럴듯한 오답)·출처 없음
- 외부세계(DB·실험·로봇) 접근 불가

에이전트 = LLM +

- 도구: 검색·계산·시뮬레이션·로봇
- 계획: 과제를 하위목표로 분해
- 메모리: 중간결과 저장·회상
- 검증: RAG 근거·재현·로그

+심화

에이전트는 LLM을 '두뇌'로 두되, 부족한 능력을 외부 도구로 위임하고, 다단계 과제를 계획으로 쪼개며, 행동의 결과를 관찰·검증하는 시스템. '아는 것'을 넘어 '행동하는' AI.

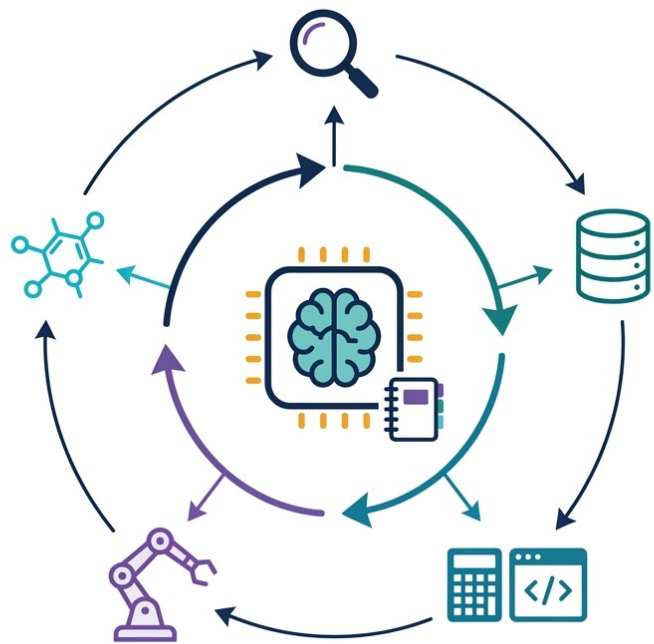
왜 '검증하는 에이전트'가 필요한가 — Galactica

- Galactica(Meta 2022): 과학 전용 LLM 데모 → 공개 3일 만에 철회
- 이유: 유창하지만 틀린 출력(할루시네이션) · 가짜 논문 · 잘못된 인용
- 교훈: '유창함(fluency) ≠ 정확성(accuracy)' — 말을 잘해도 틀린다
- 해법 방향: 도메인 특화 + RAG(근거) + 도구(계산) + 인간 · 자동 검증
→ 에이전트

⚠ **함정**

Galactica 사건은 이 세션의 출발점 — LLM 단독은 위험하다. 근거 제시(RAG) · 도구 · 검증을 붙인 '에이전트'라야 신약개발 같은 고위험 도메인에 투입할 수 있다.

LLM 에이전트 — think·act·observe



LLM 두뇌

도구 호출

결과 관찰

계획·메모리

반복

▶ LLM(추론)이 중심에서 도구(검색·DB·계산·시뮬레이션·로봇)를 호출하고 결과를 관찰해 다음 행동을 계획하는 순환. 메모리로 중간 결과를 기억 → 단일 LLM을 '행동하는 시스템'으로.

에이전트의 4대 구성요소

구성요소	역할	신약개발 예시
Brain (LLM)	추론 · 의사결정 · 언어 이해	GPT-5 · Claude 5 · Gemini 3가 절차를 설계
Tools	외부 능력 위임(계산 · 검색 · 실행)	RDKit · docking · PubMed · 로봇 합성
Planning	과제 분해 · 순서 결정 · 재계획	표적→후보→검증 단계로 분할
Memory	맥락 · 중간결과 저장 · 회상	이전 실험 결과를 다음 사이클에 반영

+심화

에이전트 = Brain(LLM) + Tools + Planning + Memory. 이 4요소가 'think(계획) · act(도구) · observe(관찰)'의 루프로 맞물려 돈다. 뒤에서 각 요소를 하나씩 심화한다.

RAG — 검색으로 근거를 붙이다

핵심 알고리즘

1. 동기 — LLM 파라미터 지식은 낮고 출처가 없음
2. 검색(Retrieval) — 질문으로 외부 지식(문헌·DB·특허) 검색
3. 증강(Augment) — 검색 결과를 컨텍스트로 프롬프트에 주입
4. 생성(Generate) — 근거에 기반해 답 + 출처 제시
5. 효과 — 환각 완화·최신성·추적가능성(provenance)

차별점 (vs 이전)

- ▶ 출처 있는 답(grounding)
- ▶ 재학습 없이 최신 지식 반영
- ▶ 신약: 논문·실험노트 근거

한계 · 주의

- 검색 품질에 의존
- 여전히 오답 가능 → 검증 필요

RAG와 도구의 결합 — 에이전트의 감각·손발

- 2교시 RAG(검색 증강) = 에이전트의 '검색 도구' 하나로 편입
- 에이전트: 검색(RAG)·계산(RDKit)·예측(ADMET)·시뮬(docking)을 상황따라 호출
- 근거(RAG) + 행동(tool) + 계획 = 자율 워크플로우(C파트)
- 모든 도구 호출은 로그로 남겨 재현·검증(4교시 provenance)

쉽게

2교시의 RAG는 이제 '검색 도구' 하나로 흡수된다. 에이전트는 검색·계산·예측·시뮬을 필요할 때 골라 쓰는 '연구자' — 그 조율이 곧 자율 워크플로우.

ReAct — 생각과 행동을 번갈아

핵심 알고리즘

1. **핵심** — 추론(Reasoning) ↔ 행동(Action)을 번갈아 생성
2. **루프** — Thought → Action → Observation → 다시 Thought
3. **Thought** — 다음에 무엇을·왜 할지 자연어로 추론
4. **Action** — 도구 호출(검색·계산·코드·시물)
5. **Observation** — 도구 결과를 관찰해 다음 추론에 반영

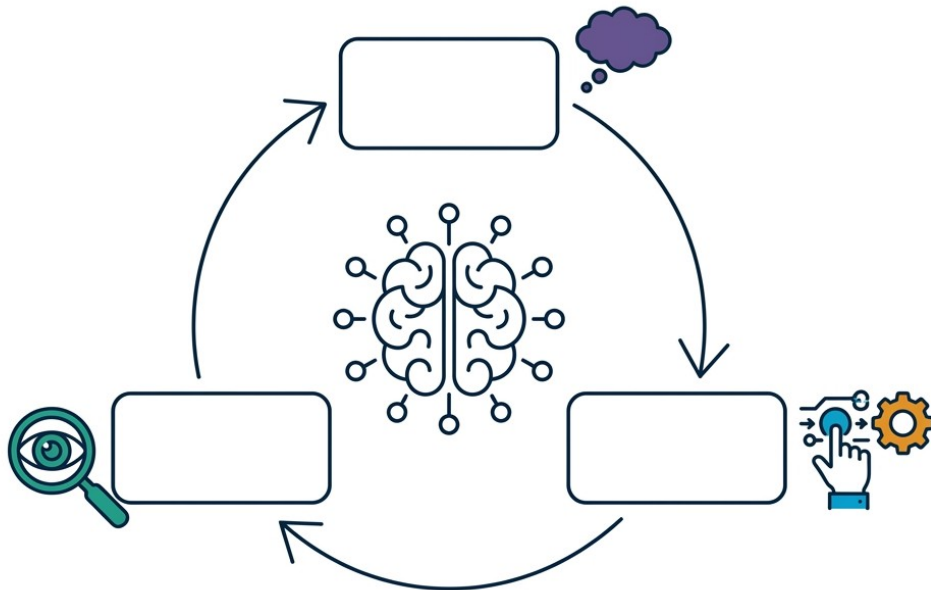
차별점 (vs 이전)

- ▶ 계획하며 외부와 상호작용
- ▶ 추론 과정이 드러나 해석·디버깅 용이
- ▶ 에이전트 루프의 원형(prompting 기법)

한계 · 주의

- 도구 오류가 추론에 전파
- 다단계일수록 실패 누적

ReAct — Thought · Action · Observation 순환



Thought

Action

Observation

반복→종료

▶ LLM이 '생각(무엇을 왜)'을 말로 남기고 → '행동(도구 호출)'하고 → '관찰(결과)'한 뒤 다시 생각으로 돌아가는 페루프. 목표 달성 또는 종료조건까지 반복. 추론이 걸으로 드러나 감사·디버깅이 쉬움.

ReAct는 Chain-of-Thought와 무엇이 다른가

Chain-of-Thought (CoT)

- 머릿속 추론만 단계적으로 서술
- 외부 도구·데이터 접근 없음
- 추론이 틀리면 정정 불가
- 수학·논리 문제에 강함
닫힌 문제

ReAct

- 추론 + 실제 행동(도구)을 교차
- 외부 관찰로 추론을 교정
- 최신·사실 정보로 grounding
- 탐색·조사형 과제에 강함
열린 문제

+심화

CoT는 '생각만' 하지만 ReAct는 '생각하고 확인'한다. 도구의 관찰로 추론 오류를 바로잡을 수 있어, 환각에 취약한 사실 기반 과제(문헌·데이터)에서 특히 유리하다.

A · ReAct 실행 예시 (신약 시나리오, 개념 도식)

Thought: 후보 X의 hERG 심장독성 위험을 확인해야 한다.

Action: `search_admet(smiles="X", endpoint="hERG")`

Observation: hERG IC50 $\approx$ 2.1 μ M (위험 구간)

Thought: 위험하다. 유사체 중 저위험 후보를 검색하자.

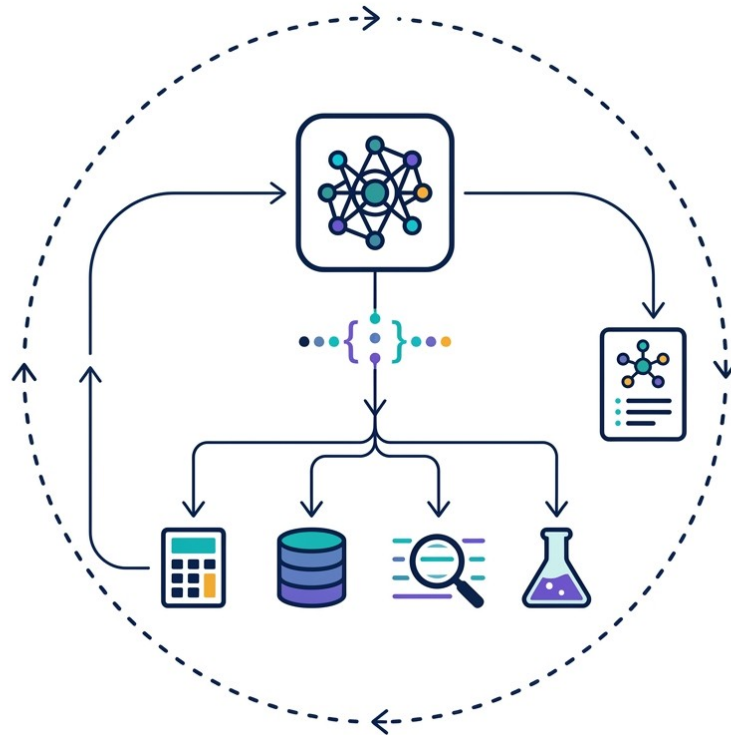
Action: `similarity_search(smiles="X", cutoff=0.7)`

Observation: 유사체 12개 반환 → ADMET 재예측 필요

Thought: 상위 3개를 docking으로 결합력 비교 후 보고.

▶ LLM이 '생각→도구호출→관찰'을 반복하며 스스로 조사·판단. 각 Action은 검증 가능한 도구 호출 — 수치는 개념 예시.

Tool use (function calling) — 도구를 손처럼



LLM

도구 선택

호출 · 실행

결과 반영

▶ LLM이 상황에 맞는 도구를 고르고 인자를 채워(function calling) 호출 → 계산기·DB·검색·시뮬레이션 결과를 받아 답에 반영. 부정확한 능력(계산·최신성)을 정확한 도구에 위임하는 것이 핵심.

Toolformer — 도구 사용을 스스로 학습

핵심 알고리즘

1. **문제** — 언제·어떤 도구를 부를지 사람이 일일이 지정?
2. **아이디어** — LLM이 스스로 도구 호출을 학습(self-supervised)
3. **방법** — 텍스트에 API 호출을 삽입해보고, 손실을 줄이는 호출만 남김
4. **도구** — 계산기·QA·검색·번역·달력 등
5. **결과** — 적은 예시로 도구 사용 능력 획득

차별점 (vs 이전)

- ▶ 도구 사용의 '학습'화
- ▶ 수작업 프롬프트 의존 감소
- ▶ 범용 도구 통합의 토대

한계 · 주의

- 학습 비용·도구 사전정의 필요
- 실행 안전성은 별도 문제

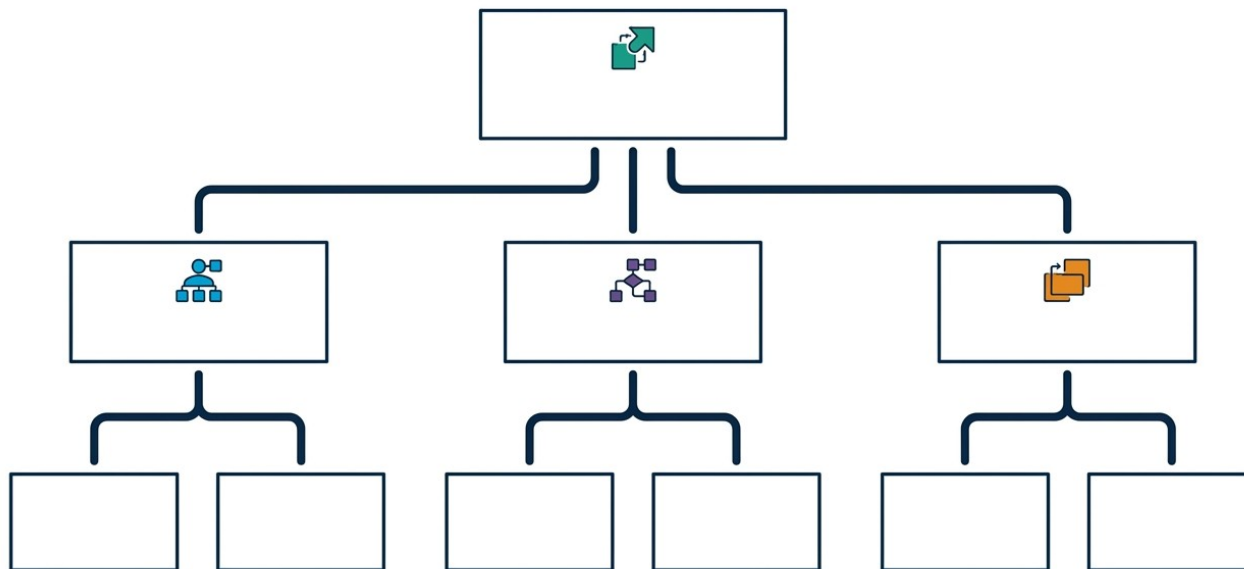
신약개발 에이전트가 쓰는 도구들

범주	도구 · API 예	역할
문헌 · 지식	PubMed · PaperQA2 · 특허 DB	근거 검색(RAG)
화학 계산	RDKit · 물성 · 유사도	구조 · 성질 계산
예측 모델	ADMET · 독성 · 활성 예측기	in-silico 스크리닝
시뮬레이션	docking · MD · retrosynthesis	결합 · 합성경로
실행	로봇 합성 · assay · 코드 실행	습식 실험 · 분석

쉽게

에이전트의 힘은 '어떤 도구를 붙였는가'에 크게 좌우된다. ChemCrow(18개+ 화학 도구)처럼 도메인 도구를 많이 · 정확히 붙일수록 유능해진다. 도구 품질 = 에이전트 품질.

Planning — 과제를 하위목표로 분해



목표

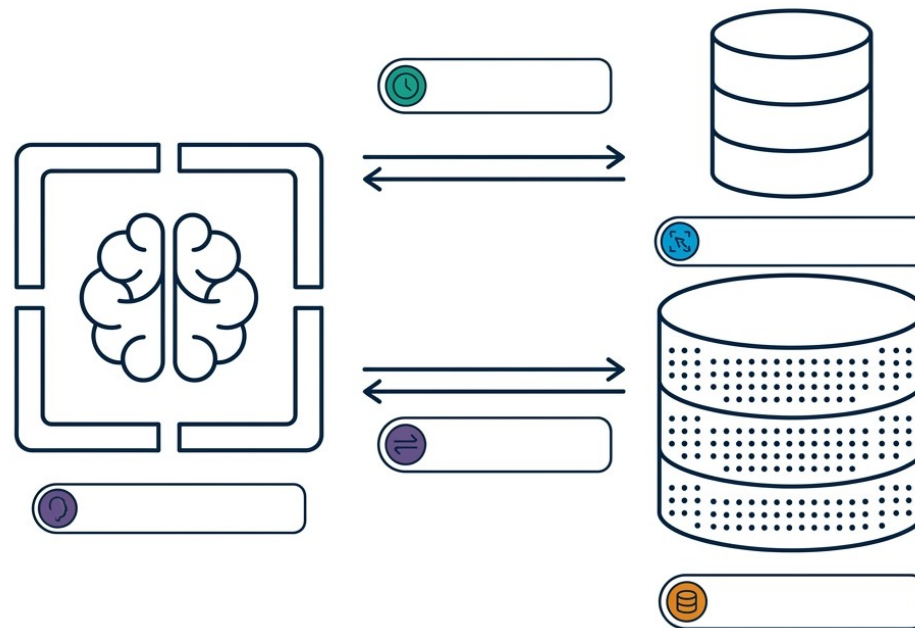
하위목표

세부 행동

재계획

▶ 복잡한 연구 과제를 한 번에 풀 수 없으므로, 큰 목표를 하위목표·세부 행동으로 분해해 순서대로 실행한다. 중간 결과가 예상과 다르면 계획을 수정(re-planning). 다단계 신약 워크플로에 필수.

Memory — 단기·장기 기억



단기(맥락)

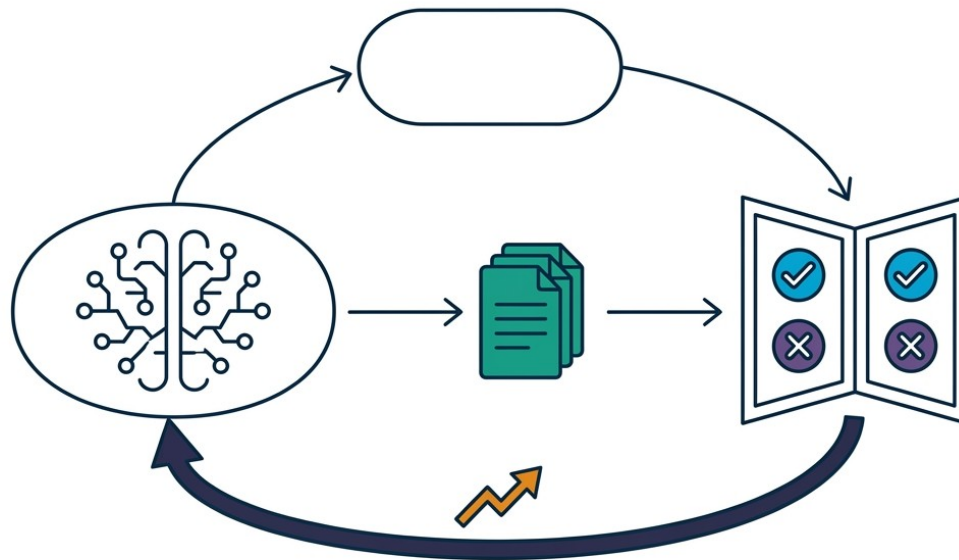
장기(벡터DB)

저장·회상

최신성

▶ 단기 메모리는 현재 대화·작업 맥락(컨텍스트 창), 장기 메모리는 과거 결과를 벡터DB에 저장했다 필요할 때 검색·회상. 이전 실험·문헌을 기억해 다음 사이클에 반영 → 누적 학습형 연구.

Reflection — 자기비평으로 개선



초안 생성

자기비평

피드백

수정 · 개선

▶ 에이전트가 자신의 출력을 스스로(또는 별도 비평 에이전트가) 검토·비평하고 그 피드백으로 다시 고치는 루프. 오류·누락을 잡아 품질을 끌어올림. multi-agent의 '검증' 역할, 4교시 하네스의 핵심.

Multi-agent — 역할을 나눠 협업·경쟁



생성

비평

순위·통합

토너먼트

▶ 여러 에이전트가 생성·반성·순위·진화·메타리뷰 등 역할을 나눠 협업하고, 후보를 토너먼트식으로 비교·정교화한다. 사람 연구팀을 흉내낸 구조로, AI co-scientist의 가설 생성이 대표 사례.

single-agent 구성 vs multi-agent

단일 에이전트 구성

- **planning:** 과제를 하위목표로 분해
- **memory:** 중간결과 저장 · 회상
- **reflection:** 자기비평 · 수정
4교시 하네스
- **tool use:** 도구 자율 선택 · 호출

multi-agent

- 역할 분담(생성 · 검증 · 순위 · 통합)
- 토너먼트식 자기개선(AI co-scientist)
- 협업 · 토론으로 복잡 과제 해결
- 검증 에이전트로 오류 억제

쉽게

복잡한 연구는 한 번에 안 됨 → 계획(분해) · 메모리(기억) · 반성(수정), 그리고 여러 에이전트의 분업 · 토론. 사람 연구팀을 흉내낸 구조. 그 '엔지니어링'은 4교시.

에이전트 설계 패턴 — 한 장 정리

패턴	작동 방식	적합 과제
ReAct	생각↔행동 교차, 관찰로 교정	탐색·조사·도구 필요
Plan-and-Execute	먼저 계획 수립 후 순차 실행	다단계·긴 워크플로
Reflexion	실패를 언어 피드백으로 재시도	반복 개선·자기교정
Multi-agent	역할 분담·토너먼트·토론	가설 생성·복잡 협업

+심화

현실 시스템은 이들을 조합한다 — 예: 계획(Plan)으로 뼈대를 잡고, 각 단계는 ReAct로 도구를 쓰며, reflection으로 검증하고, 어려운 판단은 multi-agent로. 4교시 하네스가 이 조합을 다룬다.

고정 워크플로 → 완전 자율 에이전트

수준	특징	예
고정 파이프라인	사람이 순서·도구 고정	기존 ML 스크립트
Tool-calling LLM	LLM이 도구만 골라 호출	function calling
ReAct 에이전트	생각↔행동으로 스스로 조사	ChemCrow
자율 워크플로	계획·실행·분석 페루프	Coscientist·DMTA

+심화

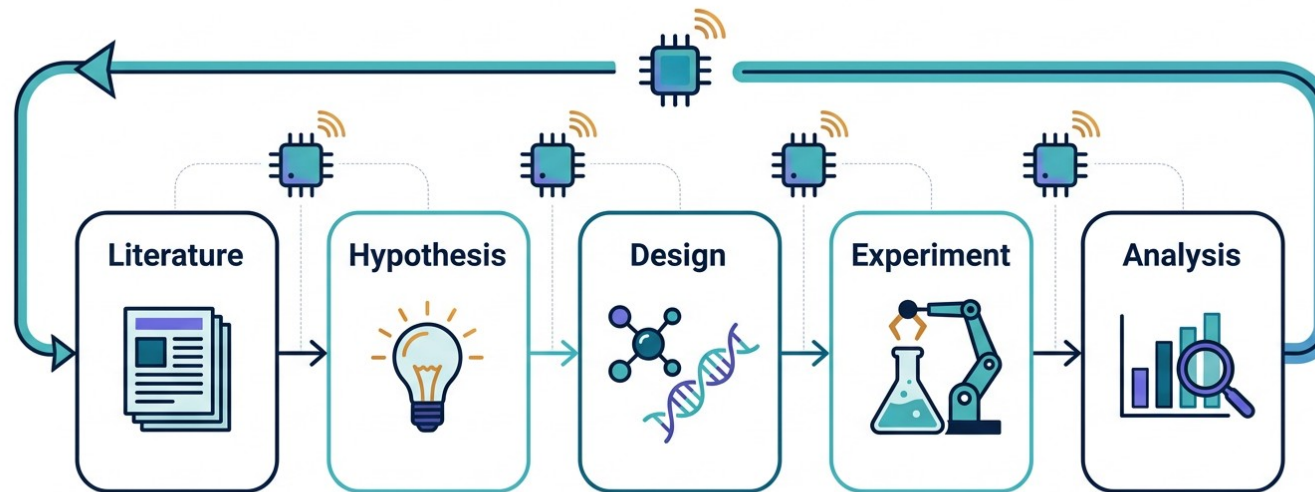
자율성이 높을수록 유연하지만 통제·검증이 어려워진다. 실무에선 '필요한 만큼만 자율' — 위험·규제가 큰 단계는 고정·게이트로, 탐색·조사는 에이전트로. 수준 선택이 곧 설계 결정.

PART B

신약개발 에이전트 사례

초기(2023) → 2025-26 다중에이전트 co-scientist로: 추적성·검증·안전이 과제

자율 과학 발견 파이프라인



문헌

가설

설계

실험

분석

▷ 문헌 검색→가설 생성→분자·단백질 설계→(로봇) 실험→데이터 분석→다시 가설로 이어지는 폐루프를 여러 에이전트가 오케스트레이션. 신약개발 전 과정을 자동화하려는 시도.

ChemCrow — 도구 사용형 화학 에이전트

핵심 알고리즘

1. **두뇌** — GPT-4를 추론 엔진으로
2. **도구** — 18개+ 전문 화학 도구(합성계획·반응예측·안전성·물성)
3. **동작** — chain-of-thought로 도구 자동 선택·호출(ReAct 계열)
4. **실행** — 로봇 합성 플랫폼과 연동(실제 합성)
5. **성과** — 곤충기피제·유기촉매 자율 합성, 신규 발색단 발견
6. **안전** — 위험물질 방지 안전 도구 내장

차별점 (vs 이전)

- ▶ LLM+전문 도구 결합의 대표
- ▶ 설계를 넘어 실제 합성까지 실행
- ▶ 안전장치를 명시적으로 포함

한계 · 주의

- 도구 품질·안전성에 의존
- dual-use(오남용) 위험

ChemCrow — 원리 · 도구 · 성과 · 한계

원리 · 도구

- LLM(GPT-4)=추론, 화학 도구=실행
- 합성계획(retrosynthesis) · 반응예측
- 분자 물성 · 안전성 조회 도구
- 도구 결과를 근거로 다음 단계 결정
도구 없이 GPT-4가 화학에 취약

성과 · 한계

- 곤충기피제 · 유기촉매 자율 합성
- 신규 발색단(chromophore) 발견
- 전문가 평가로 유용성 검증
- 한계: 도구 정확도 · dual-use · 감독 필요

+심화

ChemCrow의 교훈: '똑똑한 LLM'보다 '정확한 도구를 잘 쓰는 LLM'이 화학에서 강하다. GPT-4 단독은 SMILES · 합성에 자주 틀리지만, 전문 도구를 붙이면 실제 합성까지 해낸다.

Coscientist — 자율 화학 실험

핵심 알고리즘

1. **두뇌** — GPT-4 구동 자율 연구 시스템
2. **도구** — 인터넷·문서 검색 + 코드 실행 + 로봇 실험 API
3. **과제** — 팔라듐 촉매 교차커플링(Suzuki·Sonogashira) 자율 최적화
4. **동작** — 단일 프롬프트→절차 설계→로봇 실행→GC-MS 확인
5. **의의** — LLM이 '실험을 스스로 수행'한 대표 사례

차별점 (vs 이전)

- ▶ 설계·실행·해석 전자동
- ▶ 클라우드 로봇 실험실 통합
- ▶ 6개 과제로 능력 시연

한계 · 주의

- 좁은 반응계에 한정
- 안전·인간 감독 필요

Coscientist — 무엇을 스스로 했나

- 웹·문서를 검색해 반응 절차를 스스로 조사·설계
- Python 코드를 작성·실행하고 로봇 액체취급 장비 API를 제어
- Suzuki·Sonogashira 교차커플링을 자율 계획→실행→GC-MS로 결과 확인
- 총 6개 과제로 검색·계획·실행·분석 능력을 통합 시연

+심화

Coscientist는 '가설'을 넘어 '물리적 실험'을 자율 수행한 이정표. 다만 잘 정의된 화학 반응계에 한정되고, 안전·감독이 전제 — '완전 자율 신약개발'과는 거리가 있다.

Google AI co-scientist — 가설 생성 멀티에이전트

- Gemini 2.0 기반 multi-agent(생성·반성·순위·진화·메타리뷰 역할 분담)
- 토너먼트식 자기비평·정교화 + test-time compute 확장
- 새로운 연구가설·제안서 생성 — 항생제 내성·간섭유증 등 검증 사례
- 2025.2 공개(블로그), 이후 논문화 진행

+심화

여러 에이전트가 가설을 만들고 서로 비평·순위 매겨 발전시키는 '연구팀' 구조. 사람 대체가 아니라 '가설 생성 파트너' — 실험 검증은 사람 몫.

AI co-scientist — 멀티에이전트 구조

역할별 에이전트

- **Generation:** 가설 후보 생성
- **Reflection:** 비평·약점 지적
- **Ranking:** 토너먼트식 순위
- **Evolution · Meta-review:** 정교화·종합

핵심 메커니즘

- 자기대국(self-play)식 가설 경쟁
- test-time compute로 사고 확장
- 사람 피드백 반영(연구자 협업)
- 검증 사례로 유효성 시사
실험은 별도

+심화

'많이 생성하고, 서로 비평하고, 이긴 가설을 발전'시키는 진화적 구조. 계산을 더 쓸수록(test-time compute) 가설 품질이 오른다는 점이 특징 — 단, 최종 검증은 습식 실험.

Google 치료 파운데이션 + 자율 에이전트 (2025)

TxGemma (2025.4)

- Gemma-2 기반 2B/9B/27B 치료 특화 LM
- TDC 66개 치료 태스크(약물·표적·임상)
- Predict(예측)·Chat(대화) 두 계열
- 구 Tx-LLM 대체 세대
확인 권장(2026.9)

TxAgent / Agentic-Tx

- Gemini 2.0 기반 generalist 치료 에이전트
- 추론 + 도구 + 외부지식 호출로 자율 실행
- 치료 태스크를 스스로 계획·수행
- 파운데이션 모델 → 에이전트로 확장

+심화

Google은 치료 특화 '파운데이션 모델(TxGemma)' 위에 '자율 에이전트(TxAgent/Agentic-Tx)'를 얹는 방향 — 예측 모델을 넘어 '스스로 도구를 쓰는 치료 에이전트'. 성능 수치는 원 논문 확인 권장.

2025-26 다중 에이전트 신약 라인업

시스템	특징	근거(연도)
PharmAgents	다중 에이전트: 표적 발굴→화합물 설계	2025.3
MADD	multi-agent drug discovery 프레임워크	EMNLP 2025
DiscoVerse	추적성(traceable) 갖춘 pharma co-scientist	arXiv:2511.18259 (2025.11)
RAG-collab agents	RAG 결합 협업형 LLM 에이전트	arXiv:2502.17506 (2025.2)
AI co-scientist	가설 생성 multi-agent(생성·반성·순위)	arXiv:2502.18864 (2025.2)

+심화

2025-26 = 단일 LLM에서 '역할 분담 다중 에이전트 co-scientist'로 급속 이동. 공통 지향점은 표적→후보→검증을 잇는 협업과 '추적성(traceability)' — 어떤 근거로 결론에 이르렀는지 남기기.

FutureHouse — 과학 에이전트 스위트

도구	역할
PaperQA2	과학 문헌 RAG QA(문헌 QA superhuman 주장)
Robin	멀티에이전트: 문헌→가설→실험데이터 분석
ether0	24B 오픈웨이트 화학 추론 모델(RL 학습)
Aviary · LAB-Bench	에이전트 학습 프레임워크 · 평가 벤치마크

+심화

비영리 FutureHouse: 문헌(PaperQA2) · 가설 · 실험분석(Robin)을 잇는 과학 에이전트. 오픈 모델(ether0) · 벤치마크(LAB-Bench)까지 — 과학 에이전트 생태계의 대표 주자.

Robin — 실제 신약 재창출(repurposing) 사례

- 멀티에이전트: Crow(문헌검색) · Falcon(심층리뷰) · Finch(데이터분석) 분업
- 건성 황반변성(dAMD) 후보로 녹내장약 ripasudil(ROCK 저해제) 제안
- RPE 세포 탐식능 DMSO 대비 약 7.5배 ↑, ABCA1 상향조절 기전 규명
- 개념→논문 약 2.5개월 — 에이전트가 가설·분석을 주도

+심화

에이전트가 '기존 약물의 새 적응증'(drug repurposing)을 가설하고 실험 데이터까지 분석한 구체 사례. 다만 세포 수준 검증이며 임상은 별도 — 가설·분석 가속의 실증.

Claude for Life Sciences — 상용 과학 에이전트

- Anthropic 생명과학 특화 — 2025.10.20 발표(모델 Claude Sonnet 4.5)
- 발견 전 과정 지원: 문헌리뷰→가설→데이터분석→규제문서 초안
- MCP(Model Context Protocol) 커넥터로 외부 랩 도구·DB를 자연어 연결
- Agent Skills 제공(예: single-cell RNA-seq QC — scverse 모범관행)

쉽게

Claude for Life Sciences = '연구용 에이전트 플랫폼' — 논문을 읽고, 가설을 세우고, 데이터를 분석하고, 규제문서 초안까지. MCP로 실험실 도구를 연결해 실제 워크플로에 투입.

Claude for LS — 커넥터 & 파트너

커넥터(도구 연결)

- **Benchling**(전자연구노트 · 프로토콜)
- **10x Genomics**(단일세포 · 공간분석)
- **PubMed · Synapse · BioRender**
- **Databricks · Snowflake**
데이터 웨어하우스

파트너 · 고객

- **Sanofi · Novo Nordisk · Genmab**
- **Broad Institute · Stanford**
- **Schrödinger · EvolutionaryScale**
- **FutureHouse**
생태계 연계

+심화

핵심은 '연결성' — MCP로 실험노트(Benchling) · 유전체(10x) · 문헌(PubMed)을 자연어로 오가며 작업. 6일차 실습(Claude Code 과학 에이전트)이 바로 이 계열이며, 그 '보안 · 엔지니어링'은 4교시.

Claude for LS — Agent Skills & 성능

Agent Skills

- 도메인 절차를 스킬로 패키징
- 예: scRNA-seq QC(scverse 모범관행)
- 반복 연구작업 자동화
- 사내 프로토콜을 스킬화 가능

성능(공개 기준)

- Protocol QA 0.83 (인간 0.79)
- Sonnet 4(0.74) 대비 향상
- BixBench 등 개선 보고
- 단, 벤치마크 ≠ 실제 연구
검증 필요

+심화

Protocol QA에서 인간 기준선(0.79)을 상회(0.83) — 프로토콜 이해 · 작성에서 실무 수준 접근. 그러나 벤치마크 점수일 뿐, 실제 연구 투입 시엔 검증 · 재현이 필수(C파트).

산업 현장 배치 & 평가·서베이 (2025-26)

산업 현장 배치

- ChatInvent @ AstraZeneca — 신약 아이디어·설계 지원
- 사내 워크플로에 LLM 에이전트 투입 보고(2026.1)
- Claude for LS·Gemini for Science 등 상용 플랫폼 확산
- 실배치 세부는 확인 권장(2026.9)

평가·서베이(신뢰의 척도)

- 'Agentic Science' 서베이 — 자율과학 지형 정리
- 'Beyond SMILES' — 신약 에이전트 시스템 평가
- 단순 QA → 다단계 에이전트 과제로 평가 이동
- 벤치 점수 ≠ 실제 연구 성능
격차 인식

+심화

연구 데모를 넘어 '산업 배치'(ChatInvent@AstraZeneca)와 '체계적 평가'(Agentic Science·Beyond SMILES)가 동시에 성숙 중. 도입 확산과 함께 검증·추적성 요구도 커진다
— 성능 주장은 원문 확인 권장.

과학 AI 에이전트 경쟁 지형

기업	과학 제품 · 특징
Anthropic	Claude for Life Sciences(연구 전과정 · MCP · Agent Skills), Claude 5 계열
Google	Gemini for Science · AI co-scientist · TxGemma · TxAgent(Agentic-Tx)
OpenAI	GPT-5 계열(초기 ChemCrow · Coscientist는 GPT-4 백본)
전문 · 학계	FutureHouse · PharmAgents · MADD · DiscoVerse · Isomorphic · Recursion

+심화

빅테크(Anthropic · Google · OpenAI)와 전문 · 학계 팀이 '과학 에이전트' 시장에서 경쟁. 범용 LLM(GPT-5 · Claude 5 · Gemini 3) 위에 도메인 도구 · 커넥터 · 다중에이전트 · 검증을 엮는 방향으로 수렴 — 접근성 급상승.

신약개발 에이전트 — 한 장 비교

에이전트	주 기능	실행 수준	두뇌·연도
ChemCrow	합성계획·화학 도구	로봇 합성 실행	GPT-4 (2023)
Coscientist	실험 설계·수행	로봇 실험 실행	GPT-4 (2023)
AI co-scientist	가설 생성(multi-agent)	in-silico 가설	Gemini 2.0 (2025)
TxAgent	치료 태스크 자율	도구·외부지식	Gemini 2.0 (2025)
PharmAgents·DiscoVerse	표적→후보→검증(다중)	in-silico 협업	multi-agent (2025)
Robin	문헌→가설→분석	세포실험 분석	multi-agent (2025)
Claude for LS	연구 전과정 지원	도구 연결(MCP)	Sonnet 4.5 (2025)

+심화

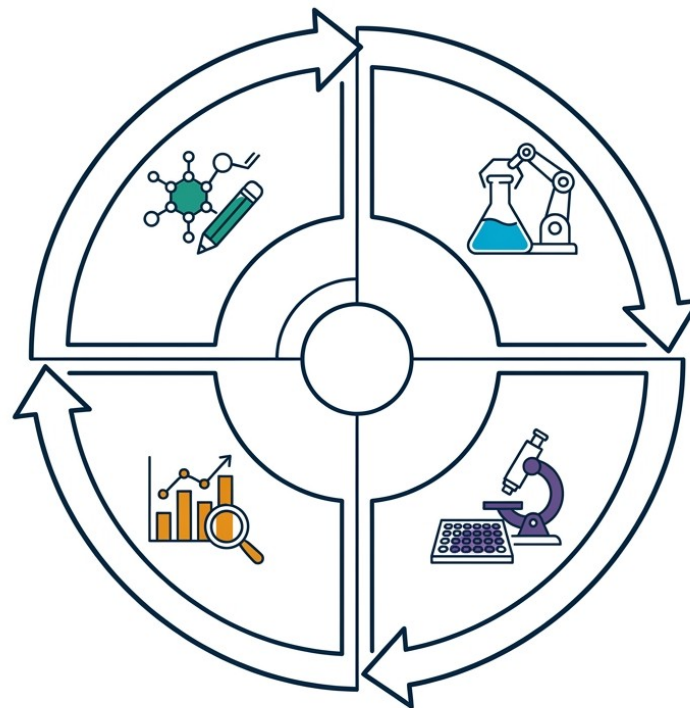
스펙트럼: 가설(AI co-scientist)→치료 태스크(TxAgent)→다중 협업(PharmAgents·DiscoVerse)→설계·실험(ChemCrow·Coscientist)→분석(Robin)→통합 플랫폼(Claude for LS). 이를 견고·안전하게 만드는 게 4교시.

PART C

자율 워크플로우 · 평가 · 한계

2025-26: 단일 LLM → 다중 에이전트 co-scientist · 추적성 · 검증 · 안전이 핵심 과제

Agentic 자율 워크플로우 — DMTA 페루프



Design

Make

Test

Analyze

▶ 에이전트가 설계(Design)-합성(Make)-시험(Test)-분석(Analyze) 사이클을 도구 호출로 자율 수행하고, 분석 결과를 다음 설계에 피드백하는 페루프. 각 단계에 검증·안전 게이트가 필요(4교시).

DMTA 각 단계의 에이전트 · 도구

단계	에이전트가 하는 일	도구 · 모델
Design	후보 분자 · 표적 설계 · 우선순위	생성모델 · MolT5 · LLM
Make	합성 경로 계획 · 로봇 합성	retrosynthesis · 로봇 API
Test	결합 · 독성 · 활성 시험	docking · ADMET · assay
Analyze	결과 해석 → 다음 사이클 계획	통계 · EDA · 메모리

+심화

에이전트가 DMTA 네 단계를 도구로 잇고, Analyze의 결과를 메모리에 남겨 다음 Design에 반영 → 사이클마다 후보가 개선되는 '자율 최적화'. 물리 실험은 자율실험실(다음 슬라이드).

자율실험실(self-driving lab) — 그리고 함정

- 에이전트(계획)+로봇(실행)+분석이 결합된 '자율 실험실'
- Coscientist·A-Lab 등 — 사람 개입 없이 실험 사이클을 반복
- A-Lab(무기재료 자율합성): 17일간 다수 신물질 합성 보고
- 단, A-Lab은 이후 '신규성·동정' 관련 정정 — 자동 판정의 함정



함정

자율실험실은 강력하지만 '자동 해석'이 틀릴 수 있다(A-Lab 정정 사례). 물리 실행을 자동화해도 '결과 판정·검증'에는 사람·독립 재현이 필요 — 자율≠무검증.

신약개발 LLM 에이전트 활용 워크플로

단계	도구·에이전트
문헌 마이닝·리뷰	PaperQA2 · Claude+PubMed · RAG
가설·표적 발굴	AI co-scientist · Robin
분자 설계·최적화	MolT5 · DrugAssist · ChemCrow
합성·실험	Coscientist · 자율실험실
데이터 분석	scRNA-seq QC · bioinformatics 에이전트
프로토콜·규제문서	Claude for LS(SOP · study protocol 초안)

+심화

LLM 에이전트는 신약개발 전 단계에 스며드는 중 — 문헌·가설·설계·합성·분석·문서, 생성모델(선행 교시)이 '분자를 만든다'면, LLM 에이전트는 '연구 과정을 운영한다'.

에이전트를 어떻게 평가하나 — 벤치마크

대표 벤치마크

- LAB-Bench: 2,400+ 생물 연구역량 문항
- BixBench: 생명정보학 에이전트
- BLURB · MedQA · PubMedQA: 생의학 QA
- Protocol QA: 프로토콜 이해

평가 관점

- 단순 QA → '에이전트형' 과제로 이동
- 문헌 · 프로토콜 · 데이터 · 그림 해석
- 도구 사용 · 다단계 성공률
- 벤치 점수 ≠ 실제 연구 성능
격차 인식

쉽게

에이전트 평가도 진화 중 — 단순 지식 QA에서 '문헌 검색 · 프로토콜 이해 · 데이터 분석' 같은 실제 연구 역량(LAB-Bench)으로. 단 벤치와 현실의 격차는 상존한다.

오류 전파 — 다단계 자율의 함정

- 환각·오류가 다단계로 전파·누적(도구 오호출·잘못된 계획)
- 단계별 성공률 0.9라도 10단계면 $0.9^{10} \approx 0.35$ 로 급락
- 도구 결과를 무비판 수용하면 오류가 '사실'로 굳어짐
- 대응: 단계별 검증 게이트·reflection·human-in-the-loop

⚠ 함정

자율 다단계일수록 실패가 곱해진다. 각 단계에 검증·롤백·인간 게이트가 없으면 '자율'이 곧 '자동 오류 증폭'이 된다 — 4교시 하네스의 존재 이유.

검증·재현성 — AI 산출은 '가설'

리스크	내용·대응
할루시네이션	그럴듯한 오답 → RAG·도구·인간검증
과학적 검증	AI 산출=가설 → 습식실험·독립 재현 확증
재현성	프롬프트·seed·모델버전·provenance 고정
벤치마크 격차	벤치 점수 ≠ 실제 연구 성능



함정

Galactica(그럴듯한 오류)·A-Lab(신규성 정정) 사례 → AI 산출은 '가설'이지 '결론'이 아니다. 전공·임상 활용 시 RAG·도구·인간 검증·재현이 필수.

보안·거버넌스 — 4교시 예고

- 도구 권한·API 키·기밀 데이터 노출 위험(에이전트가 외부와 상호작용)
- 프롬프트 인젝션·도구 오남용·dual-use(위험물질 설계 등)
- 규제·임상 문서는 사람이 최종 책임 — 자동 제출 금지
- 해법: 권한 최소화·감사로그·게이트·샌드박스(4교시 하네스)

+심화

에이전트가 강력해질수록 '무엇을·어디까지 하게 둘 것인가'가 핵심. 권한·격리·감사·인간 승인 같은 통제(거버넌스)가 없으면 신약개발 현업엔 못 쓴다 — 그 엔지니어링이 4교시.

에이전트 실패 모드 — 무엇을 감시하나

실패 모드	증상	완화책
잘못된 도구 선택	엉뚱한 API · 인자 호출	도구 설명 · 검증 게이트
도구 결과 환각	없는 결과를 지어냄	실제 반환값 강제 · 로그
무한 루프	같은 행동 반복 · 미종료	step 상한 · 종료조건
목표 이탈(drift)	원 과제에서 벗어남	계획 재확인 · 감독



함정

에이전트는 '조용히' 실패한다 — 그럴듯한 로그를 남기며 틀린다. 그래서 도구 반환값 검증 · step 상한 · 감사로그 · 인간 게이트가 필수(4교시 하네스가 이 감시를 체계화).

현직자를 위한 실무 교훈

1

LLM 에이전트는 '가설·초안 생성기' — 결론이 아니라 출발점으로

2

반드시 RAG·도구로 근거를 붙이고 출처를 확인(할루시네이션 방지)

3

생성물·분석은 습식실험·독립 재현으로 검증

4

프롬프트·모델버전·seed·도구 로그를 기록(재현성·규제 대비)

5

도메인 도구·플랫폼(Claude for LS·MCP)으로 현업 워크플로에 통합

3교시 정리

- 1 에이전트 = LLM(추론) + 도구 + 계획 + 메모리 + 검증
- 2 ReAct(생각→행동→관찰) · tool use · Toolformer · reflection · multi-agent
- 3 초기(ChemCrow · Coscientist, 2023) → 2025-26 다중에이전트(AI co-scientist · TxAgent · PharmAgents · DiscoVerse) · Claude for LS
- 4 핵심 전환: 단일 LLM → 다중 에이전트 co-scientist. 추적성 · 검증 · 안전이 과제
- 5 다음: 에이전트를 '견고 · 안전하게' 만드는 하네스 엔지니어링
4교시

4교시 예고

에이전트를 견고·안전하게 — 하네스 엔지니어링

컨텍스트 엔지니어링 · 서브에이전트 · 메모리/LLM wiki · MCP · 평가 · Agentic 보안 통제

에이전트를 '만들 수 있다'에서 '믿고 운영할 수 있다'로.

주요 출처 (3교시) — 1/2

- 에이전트 원리: Yao et al. 2023 (ReAct, ICLR, arXiv:2210.03629); Schick et al. 2023 (Toolformer, NeurIPS, arXiv:2302.04761).
- 원리(계속): Lewis et al. 2020 (RAG, NeurIPS, arXiv:2005.11401); Wei et al. 2022 (Chain-of-Thought); Shinn et al. 2023 (Reflexion); Wang et al. 2024 (agent survey).
- 사례 — 화학: M. Bran et al. 2024 (ChemCrow, Nat Mach Intell 6:525–535, 10.1038/s42256-024-00832-8).
- 사례 — 실험: Boiko et al. 2023 (Coscientist, Nature 624:570–578, 10.1038/s41586-023-06792-0).
- 배경: Taylor et al. 2022 (Galactica, arXiv:2211.09085); Heaven 2022 (MIT Tech Review).

주요 출처 (3교시) — 2/2

- 가설·문헌 에이전트: Google 2025 (AI co-scientist, arXiv:2502.18864); Skarlinski et al. 2024 (PaperQA2, arXiv:2409.13740).
- 치료 파운데이션·에이전트: TxGemma (arXiv:2504.06196, 2025.4); TxAgent/Agentic-Tx (Google, Gemini 2.0).
- 다중에이전트 신약(2025-26): PharmAgents (2025.3); MADD (EMNLP 2025); DiscoVerse (arXiv:2511.18259); RAG-collab agents (arXiv:2502.17506).
- 산업배치·평가/서베이: ChatInvent@AstraZeneca (2026.1 보고); Agentic Science 서베이 (arXiv:2508.14111); Beyond SMILES (arXiv:2602.10163).
- FutureHouse·상용·실험실: Robin (arXiv:2505.13400); LAB-Bench (arXiv:2407.10362); Claude for LS (2025.10.20); A-Lab (Nature) + 정정.
- * 확인 권장(2026.9): TxAgent 서지·수치, ChatInvent 실패치 세부, AI co-scientist Nature 게재, Robin 정식 서지·일부 수치(프리프린트 기준).